Step-by-Step Implementation Plan for Adaptive Unlearning for Dynamic Fairness in Changing Data Distributions

Research Methodology

1. Problem Formulation

Let the deployed predictor be

$$f_{\theta_t} : \mathcal{X} \to \hat{\mathcal{Y}},$$

where θ_t denotes model parameters at time t. Data arrive as a stream

$$\mathcal{D}^{(t)} = \{(x_i, y_i, a_i)\}_{i=1}^{n_t},$$

where $x_i \in \mathcal{X}$ are features, $y_i \in \mathcal{Y}$ are labels, and $a_i \in \mathcal{A}$ is the protected attribute (or inferred group membership if explicitly allowed in the setting).

The objective is to design an adaptive framework that:

1. continuously monitors fairness over time,
2. detects fairness degradation under distribution shift,
3. identifies the responsible data points, batches, or features,
4. selectively unlearns their influence,
5. preserves task utility and avoids catastrophic forgetting,
6. maintains a complete audit trail for accountability.

This can be formulated as a constrained online optimization problem:

$$\min_{\theta_t} \ \mathcal{L}_{task}(\theta_t) + \lambda_u \mathcal{L}_{unlearn}(\theta_t) + \lambda_s \mathcal{L}_{stability}(\theta_t)$$

subject to

$$\mathcal{F}(\theta_t; W_t) \leq \delta_f,$$

where:

- $\mathcal{L}_{task}$ is predictive loss,
- $\mathcal{L}_{unlearn}$ enforces removal of harmful influence,

- $\mathcal{L}_{stability}$ preserves performance on retained knowledge,
- $\mathcal{F}(\theta_t; W_t)$ is a fairness violation measure computed on a recent sliding window W_t,
- δ_f is an acceptable fairness threshold.

Typical fairness metrics include:

$$\text{Statistical Parity Difference (SPD)} = P(\hat{Y} = 1 \mid A = 0) - P(\hat{Y} = 1 \mid A = 1),$$

$$\text{Equal Opportunity Difference (EOD)} = P(\hat{Y} = 1 \mid Y = 1, A = 0) - P(\hat{Y} = 1 \mid Y = 1, A = 1).$$

2. Overall System Architecture

The system should contain six tightly connected modules:

1. **Streaming Data Buffer**: stores recent data in a sliding window and summarized historical statistics.
2. **Online Fairness Monitor**: tracks utility and fairness metrics over time.
3. **Drift and Bias Detector**: identifies distribution shift and fairness re-emergence.
4. **Bias Attribution Engine**: localizes data points, batches, or features contributing to unfairness.
5. **Selective Unlearning Engine**: removes harmful influence without full retraining.
6. **Audit and Governance Module**: logs why, when, and how unlearning occurred.

A simple processing pipeline is:

stream input $\rightarrow$ monitor $\rightarrow$ detect violation $\rightarrow$ attribute cause $\rightarrow$ selective unlearning $\rightarrow$ re-evaluate.

3. Step-by-Step Methodology

Step 1: Choose Base Learning Setting

Start with a setting that supports incremental updates and approximate unlearning:

- logistic regression,
- online linear models,
- shallow neural networks,

- mini-batch updated deep models with parameter-efficient adaptation layers.

For initial research validation, it is recommended to begin with:

1. a convex model (e.g., logistic regression) for provable analysis, and
2. a non-convex model (e.g., MLP or small Transformer tabular model) for practical validation.

Reason. A convex model makes influence estimation and certified unlearning easier, while a neural model demonstrates applicability in realistic deployment.

Step 2: Define the Streaming Setting and Windowing Strategy

Maintain:

- a **recent window** W_t of size w for fairness monitoring,
- a **reference buffer** R_t representing historical stable data,
- optionally, a **replay memory** M_t containing utility-critical examples.

At each time t, update:

$$W_t = \{(x_i, y_i, a_i)\}_{i=t-w+1}^{t}.$$

You may also use:

- fixed-size sliding windows,
- exponentially decayed windows,
- adaptive windows that expand or contract based on detected drift.

Implementation decision. Use a sliding window for fairness metrics and a compact replay buffer for utility preservation.

Step 3: Build the Online Fairness Monitoring Module

For every update interval (per sample, per batch, or every k batches), compute:

- accuracy / F1 / AUROC,
- SPD,
- EOD,
- calibration gap by group,
- optionally, subgroup fairness for intersectional groups.

Define a composite fairness alert score:

$$S_t = \alpha_1 |\mathrm{SPD}_t| + \alpha_2 |\mathrm{EOD}_t| + \alpha_3 |\mathrm{CalibrationGap}_t|.$$

Trigger a fairness alert when:

$$S_t > \tau_f \quad \text{or} \quad |\mathcal{F}_t - \mathcal{F}_{t-1}| > \tau_\Delta.$$

To reduce false alarms:

- smooth metrics using exponential moving averages,
- require violations to persist for m consecutive windows,
- use confidence intervals or bootstrap uncertainty estimates.

Step 4: Detect Distribution Shift and Bias Re-Emergence

Bias may reappear because:

- the data distribution changes,
- a previously neutral feature becomes a proxy for protected attributes,
- new biased batches are injected,
- label quality differs across groups.

Implement two complementary detectors:

(a) Covariate / concept drift detector. Measure changes in:

- feature distributions $P_t(X)$,
- label distributions $P_t(Y)$,
- conditional performance $P_t(\hat{Y}|Y, A)$.

Possible detectors include:

- Population Stability Index (PSI),
- KL divergence or Jensen-Shannon divergence,
- Kolmogorov–Smirnov tests,
- ADWIN, DDM, or EDDM for stream drift.

(b) Fairness drift detector. Track fairness trajectories:

$$\Delta\mathcal{F}_t = \mathcal{F}(W_t) - \mathcal{F}(W_{t-1}).$$

A fairness violation is declared if there is both:

1. sustained fairness degradation, and
2. evidence of drift or biased incoming data.

This two-level trigger prevents unnecessary unlearning when fairness fluctuations are only due to random noise.

Step 5: Attribute the Source of Unfairness

Once a fairness violation is detected, the main challenge is to identify the most likely causes without exhaustive search.

Use a three-level attribution strategy:

Level 1: Batch-level screening. Assign each recent batch B_j a harmfulness score:

$$H(B_j) = \mathcal{F}(W_t) - \mathcal{F}(W_t \setminus B_j),$$

approximated efficiently by influence estimates rather than full recomputation.

Level 2: Point-level attribution. For candidate batches only, estimate the influence of each point z_i on fairness:

$$I_{\mathcal{F}}(z_i) \approx -\nabla_\theta \mathcal{F}(\theta_t)^\top H_\theta^{-1} \nabla_\theta \ell(z_i, \theta_t),$$

where H_θ is the Hessian (or its approximation) and ℓ is the training loss.

Interpretation:

- if $I_{\mathcal{F}}(z_i)$ is large and positive, the point contributes strongly to unfairness,
- points with top-k influence become unlearning candidates.

Level 3: Feature-level attribution. Use SHAP, gradient attribution, or counterfactual sensitivity analysis to detect features that have become unfair proxies:

$$C_j = \text{Influence of feature } x_j \text{ on fairness violation.}$$

This is useful when bias re-emergence is due to feature correlations rather than a few isolated examples.

Practical recommendation. Use a hierarchical pipeline:

$$\text{batch filter} \rightarrow \text{point influence ranking} \rightarrow \text{feature attribution},$$

since full point-wise Shapley computation for all stream data is too expensive.

Step 6: Define the Unlearning Unit

The framework should support multiple unlearning granularities:

1. **Point unlearning**: remove individual harmful training examples.
2. **Batch unlearning**: remove or discount an entire incoming batch.
3. **Feature unlearning**: suppress or de-correlate a feature that acts as a bias proxy.
4. **Group-conditional unlearning**: reduce the contribution of harmful patterns tied to a subgroup-specific drift.

Define the unlearning candidate set:

$$U_t = \{z_i \in W_t \cup R_t : I_{\mathcal{F}}(z_i) > \tau_u\}.$$

You may also choose a budgeted version:

$$U_t = \arg \max_{S \subseteq \mathcal{C},\ |S| \leq b} \sum_{z_i \in S} I_{\mathcal{F}}(z_i),$$

where $\mathcal{C}$ is the candidate pool and b is the unlearning budget.

Step 7: Design the Selective Unlearning Mechanism

This is the core algorithmic contribution. A useful strategy is to compare and integrate three mechanisms.

Mechanism A: Influence-based parameter correction. Approximate removal of a point's effect without retraining:

$$\theta_t^{(-U)} \approx \theta_t + \frac{1}{n} H_\theta^{-1} \sum_{z_i \in U_t} \nabla_\theta \ell(z_i, \theta_t).$$

Advantages:

- fast,
- principled,
- suitable for moderate models with Hessian approximations.

Mechanism B: Gradient reversal / anti-fine-tuning. Perform a few update steps to *forget* selected examples:

$$\theta \leftarrow \theta + \eta \sum_{z_i \in U_t} \nabla_\theta \ell(z_i, \theta) - \eta \lambda_r \sum_{z_j \in M_t} \nabla_\theta \ell(z_j, \theta),$$

where the first term removes harmful influence and the second term preserves performance on retained data.

Mechanism C: Weighted re-optimization on a small buffer. Instead of deleting examples, reweight them:

$$\mathcal{L}_{adapt} = \sum_{z_i \in W_t \cup M_t} w_i \ell(z_i, \theta),$$

where harmful points get low or zero weight and fairness-preserving support data get high weight.

Recommended primary approach. A hybrid selective unlearning method can be proposed:

1. use influence estimation to identify candidate points,
2. apply a small number of anti-fine-tuning steps,
3. regularize with replay memory and distillation to preserve utility.

Step 8: Add Utility Preservation Constraints

To avoid catastrophic forgetting, add one or more of the following:

(a) Replay memory. Maintain a representative buffer M_t of useful historical examples.

(b) Knowledge distillation. Let the pre-unlearning model be f_{θ_t} and the updated model be $f_{\theta'_t}$. Minimize:

$$\mathcal{L}_{KD} = \mathbb{E}_{x \in M_t}\Big[D_{KL}(f_{\theta_t}(x) \,\|\, f_{\theta'_t}(x))\Big].$$

(c) Parameter regularization. Penalize large deviation from previous parameters:

$$\mathcal{L}_{reg} = \|\theta'_t - \theta_t\|_2^2.$$

(d) Fisher-based protection. Protect important parameters:

$$\mathcal{L}_{EWC} = \sum_k F_k(\theta'_k - \theta_{t,k})^2,$$

where F_k is parameter importance.

Final unlearning objective.

$$\mathcal{L}_{total} = \mathcal{L}_{forget}(U_t) + \lambda_1 \mathcal{L}_{task}(M_t) + \lambda_2 \mathcal{L}_{KD}(M_t) + \lambda_3 \mathcal{L}_{reg}.$$

Step 9: Add Fairness-Aware Stopping Conditions

After each selective unlearning episode, re-evaluate on the current window W_t.

Stop adaptation when:

1. fairness violation drops below threshold:

$$\mathcal{F}(W_t) \leq \delta_f,$$

2. utility remains acceptable:

$$\mathrm{Acc}(W_t) \geq \delta_{acc},$$

3. update cost is below a budget,
4. maximum number of unlearning iterations is reached.

A multi-objective acceptance rule can be:

$$J = \beta_1 \Delta\mathrm{Fairness} - \beta_2 \Delta\mathrm{Error} - \beta_3 \mathrm{Cost}.$$

Accept the unlearning action only if $J > 0$.

Step 10: Maintain an Audit Trail

For accountability, every adaptation must be logged with:

- timestamp,
- violated fairness metric(s),
- drift indicators,
- attributed sources (batch, points, features),
- unlearning action applied,
- before/after fairness and utility,
- computational time and memory cost.

Store an audit record:

$$\mathcal{A}_t = \big(t, \mathcal{F}_{before}, \mathcal{F}_{after}, U_t, \Delta\mathrm{Acc}, \Delta\mathrm{Cost}, \mathrm{reason}\big).$$

This module is essential for:

1. reproducibility,
2. regulatory compliance,
3. debugging unfair adaptation cycles,
4. explaining why a deployed model changed over time.

4. Proposed End-to-End Algorithm

A possible algorithm is:

1. Train an initial model f_{θ_0} on initial data.
2. Initialize fairness monitor, drift detector, replay memory, and audit log.
3. For each incoming batch B_t:
   (a) make predictions on B_t,
   (b) update sliding window W_t,
   (c) compute fairness metrics and utility metrics,
   (d) run drift detection,
   (e) if no persistent fairness violation is found, continue online learning,
   (f) otherwise:
      i. score recent batches for unfairness contribution,
      ii. rank candidate points/features using influence or SHAP,
      iii. select unlearning set U_t under a budget,
      iv. perform selective unlearning with utility-preserving regularization,
      v. re-evaluate fairness and utility,
      vi. if improved, commit update; else rollback,
      vii. log the entire event in the audit trail.

5. Suggested Pseudocode

```
Input:
    Initial model f_theta0
    Stream of batches {B_t}
    Fairness threshold delta_f
    Unlearning budget b
    Replay buffer M
Initialize:
    theta <- theta0
    W <- empty sliding window
    AuditLog <- []

for each batch B_t:
    Predict on B_t using f_theta
    Update W with B_t
    Compute utility metrics U_t
    Compute fairness metrics F_t
    Compute drift score D_t
```

```
if FairnessViolation(F_t, delta_f) and Persistent(F_t) and DriftDetected(D_t):
    CandidateBatches <- ScreenRecentBatches(W)
    CandidatePoints <- RankByInfluence(CandidateBatches, theta)
    CandidateFeatures <- RankByFairnessAttribution(W, theta)

    Uset <- SelectTopHarmful(CandidatePoints, CandidateFeatures, budget=b)

    theta_new <- SelectiveUnlearn(theta, Uset, M)

    Evaluate F_new, U_new on W

    if Accept(theta_new, F_new, U_new, cost):
        theta <- theta_new
        Log(AuditLog, t, F_t, F_new, Uset, U_t, U_new, cost)
    else:
        Rollback()

else:
    theta <- StandardIncrementalUpdate(theta, B_t)
    UpdateReplayMemory(M, B_t)
```

6. Experimental Design

6.1 Datasets

Use both synthetic and real-world streaming datasets.

Synthetic data. Generate controlled streams where:

- the relation between features and protected attributes changes over time,
- biased batches are injected at chosen time steps,
- class imbalance evolves,
- subgroup label noise increases in specific periods.

This allows precise evaluation of:

- detection quality,
- attribution accuracy,
- unlearning effectiveness.

Real-world data candidates. Examples include:

- online lending / credit approval,
- dynamic healthcare risk prediction,

- recidivism-style tabular decision streams,
- employee hiring / attrition prediction streams,
- ad allocation or recommendation streams.

To simulate streaming, partition data chronologically or create pseudo-temporal segments.

6.2 Baselines

Compare against:

1. **No intervention** (plain online learning),
2. **Periodic full retraining**,
3. **Static fairness-aware retraining**,
4. **Single-shot unlearning** without online monitoring,
5. **Reweighting / resampling** methods without unlearning,
6. **Drift adaptation only** without fairness correction.

6.3 Evaluation Metrics

Measure three dimensions.

(a) Fairness performance.

- average SPD over time,
- average EOD over time,
- maximum fairness violation,
- fairness stability (variance of fairness metrics),
- recovery time after fairness drift.

(b) Utility performance.

- accuracy / F1 / AUROC,
- group-wise performance,
- calibration,
- forgetting score on retained knowledge.

(c) Efficiency performance.

- adaptation latency,
- memory usage,
- number of retraining-equivalent operations,
- cost per fairness recovery event.

Additional metric: attribution quality. If synthetic harmful batches are known, evaluate:

$$\text{Precision@k}, \quad \text{Recall@k}$$

for source identification.

7. Core Research Contributions to Target

Your implementation should aim to contribute in the following ways:

1. **A fairness-drift trigger**: a principled online criterion for deciding when unlearning should happen.
2. **A scalable attribution method**: identifying unfairness-causing data efficiently in streams.
3. **A selective unlearning algorithm**: removing harmful influence without full retraining.
4. **A utility-preserving mechanism**: replay, distillation, and regularization to prevent catastrophic forgetting.
5. **An accountability layer**: transparent logs of adaptive fairness interventions.

8. Recommended Phased Implementation Roadmap

Phase 1: Formalization and Simulation

- Define fairness metrics and thresholds.
- Create synthetic streaming datasets with controlled drift and bias injection.
- Implement online monitor and drift detector.

Phase 2: Attribution Prototype

- Implement batch-level harmfulness screening.
- Add point-level influence estimation.
- Add feature-level attribution for proxy detection.

Phase 3: Selective Unlearning Engine

- Implement approximate unlearning via anti-fine-tuning or influence correction.
- Add replay memory and distillation-based utility preservation.
- Implement rollback if fairness improves but utility collapses.

Phase 4: Real-World Validation

- Apply the method to one or more real datasets with temporal structure.
- Compare against periodic retraining and fairness-aware retraining baselines.

Phase 5: Governance and Analysis

- Build audit logs and visualization of fairness trajectories.
- Analyze which kinds of drift are best handled by selective unlearning.
- Perform ablation studies.

9. Suggested Ablation Studies

To make the paper strong, include ablations on:

- sliding window size w,
- fairness threshold δ_f,
- attribution method (influence vs SHAP vs leave-one-batch-out),
- unlearning unit (point vs batch vs feature),
- replay buffer size,
- distillation strength,
- unlearning budget b,
- drift detector choice.

10. Key Challenges and How to Address Them

Challenge 1: False fairness alarms.

- Use persistent violation checks.
- Use confidence intervals and smoothed metrics.

Challenge 2: Expensive point-level attribution.

- Perform hierarchical attribution: batch first, point second.
- Use Hessian-vector product approximations instead of exact Hessians.

Challenge 3: Utility collapse after unlearning.

- Use replay memory, distillation, and parameter regularization.
- Enforce acceptance criteria before committing updates.

Challenge 4: Oscillation between fairness and utility.

- Formulate acceptance as a multi-objective criterion.
- Add cooldown intervals between successive unlearning actions.

Challenge 5: Accountability in adaptive systems.

- Log all actions, causes, thresholds, and performance deltas.

11. Final Proposed Method Summary

A concise summary of the proposed method is:

> Develop an online fairness-aware machine unlearning framework that monitors fairness on recent data windows, detects fairness drift under changing distributions, attributes unfairness to harmful batches/points/features using scalable influence-based methods, selectively removes their effect through approximate unlearning rather than full retraining, and preserves utility through replay and distillation, while keeping a full audit trail of interventions.

12. Possible Name for the Method

A good working title for the method could be:

AFU-Stream: Adaptive Fairness Unlearning for Streaming Data

or

DAFU: Dynamic Adaptive Fairness Unlearning.

13. Expected Outcome

If implemented successfully, the framework should:

- detect fairness degradation earlier than periodic audits,
- recover fairness faster than full retraining baselines,
- use significantly lower computation than retraining from scratch,
- maintain competitive predictive accuracy,
- provide transparent logs for every fairness intervention.

14. One-Sentence Research Thesis

> *Selective, trigger-based adaptive unlearning can stabilize fairness in non-stationary machine learning systems more efficiently than periodic retraining, while preserving utility and enabling accountable deployment.*